

Department of Computer Science and Engineering

Assignment Questions

Course Code & Name : CSA0603 –Design and Analysis of Algorithms

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.E/B.Tech-Computer Science and Engineering
Course Code & Course Name	CSA0603-Design and Analysis of Algorithms
Academic Year / Batch	2026-Regular
Faculty Name	Dr.P.Solainayagi
Assignment Title	Analytical Study and Implementation of Brute Force Algorithms for Selection Sort and Sequential Search
Date of Issue	01-09-2026
Date of Submission	02-09-2025
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply Brute Force, Searching, Sorting, Closest Pair, Convex Hull.
Bloom's Taxonomy Level	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education: Develops analytical thinking, programming, and computational problem-solving skills. SDG 9 – Industry, Innovation and Infrastructure – Promotes innovation and development of computational techniques for solving real-world spatial and engineering problems.
Industry / Societal Relevance	Promotes innovation and development of computational techniques for solving real-world spatial and engineering problems.

B. Assignment Problem / Challenge

Given the set of 12 numbers $A = \{42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21\}$, apply the Brute Force technique to solve the Sorting and Searching problems. For Selection Sort, repeatedly identify the smallest element from the unsorted portion and place it in its correct position. Show the major intermediate steps, determine the sorted sequence, develop suitable algorithm/pseudocode, and analyze the number of comparisons and swaps performed. For Sequential Search, search for the key value 68 by checking elements one by one from the beginning of the list.

C. Problem Statement

Identify the position of the key, show the comparisons performed, and develop suitable algorithm/pseudocode. Finally, compare the two Brute Force approaches in terms of strategy, basic operations, computational cost as n increases, limitations for large datasets, and possible improvements using more efficient algorithms. For Selection Sort, derive the total number of comparisons using the summation $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$. Count the number of swaps performed for the given input and distinguish comparisons from data movements. For Sequential Search, determine the number of comparisons required to find 68 and discuss best-case, average-case, and worst-case behavior. Express the time complexity of Selection Sort and Sequential Search using Big-O, Big- Ω , and Big- Θ notations. Discuss how the algorithms behave when the input size increases and identify the situations in which their brute-force nature becomes inefficient. Suggest efficient alternatives such as Merge Sort/Quick Sort for sorting and Binary Search for searching, stating the conditions required for their use.

D. Requirements and Constraints

E. Use a publicly available real-world numeric dataset containing enough records to test different input sizes from 10^3 to 10^6 records. Use the same dataset subsets and the same computational environment for a fair comparison. Implement and evaluate the following where applicable: (1) Brute Force Selection Sort, (2) an efficient Divide-and-Conquer sorting algorithm such as Merge Sort, and (3) a Hybrid sorting algorithm such as IntroSort or a Merge/Insertion hybrid. For searching, implement Brute Force Sequential Search and an efficient search method such as Binary Search after sorting the data. Record execution time and operation counts for each input size. Clearly document the dataset source, preprocessing, hardware/software environment, implementation language, hybrid threshold, and all assumptions. Analyze how performance changes as n increases. Conclude by relating the measured results to the theoretical complexities. Explain why brute-force methods are attractive for simplicity and small inputs, but become costly as n increases. Discuss whether the experimental results support the expected $\Theta(n^2)$ behavior of Selection Sort and $\Theta(n)$ worst-case behavior of Sequential Search, and identify the practical benefits of divide-and-conquer and hybrid approaches.

F. Student Work

1. Problem Understanding and Formulation

- Problem analysis and identification of inputs/outputs.
- Brute Force algorithm/pseudocode for Selection Sort and Sequential Search.
- Step-by-step Selection Sort passes and search comparisons.
- Final sorted sequence and Sequential Search result for key 68.
- Count of comparisons, swaps/movements, and search operations.
- Implementation of Selection Sort, Merge Sort, and a Hybrid Sort for experimental comparison.
- Implementation of Sequential Search and Binary Search for search comparison.
- Time and space complexity analysis using asymptotic notations.
- Experimental graphs: input size vs execution time and input size vs operation count.
- Comparative analysis, assumptions, limitations, and possible improvements.
- Final conclusion based on both theoretical and experimental observations.

2. Application of Course Knowledge

Identify and apply the relevant:

- Principles
- Theories
- Mathematical models
- Algorithms
- Engineering concepts
- Scientific concepts
- Design methodologies

Show necessary calculations, derivations or reasoning.

3. Solution / Design / Methodology

Develop the proposed solution. Depending upon the nature of the course, this may include:

- Design
- Algorithm
- Model
- Circuit
- Architecture
- Experiment
- Process
- Prototype
- Simulation
- Case analysis
- Data analysis
- Mathematical solution

Where appropriate, consider **more than one possible solution**.

4. Use of Modern Tools

Use appropriate tools wherever relevant.

Examples:

Python, MATLAB, TensorFlow, cloud platforms, Git, simulation tools

Students shall provide appropriate evidence of tool usage and outputs.

5. Results and Validation

Present the results using appropriate:

- Tables
- Graphs
- Simulation outputs

- Experimental observations
- Screenshots
- Performance metrics
- Statistical analysis
- Test results

Validate whether the proposed solution satisfies the stated requirements.

6. Analysis and Engineering Decision

Interpret the results.

Where applicable:

- Compare alternatives.
- Identify advantages and limitations.
- Discuss trade-offs.
- Explain why the final solution was selected.
- Justify the decision using quantitative or qualitative evidence.

7. Broader Considerations

Where relevant to the problem, discuss the impact of the proposed solution with respect to:

- Sustainability
- Environment
- Society
- Safety
- Ethics
- Economics
- Accessibility

- Professional responsibility

8. Conclusion

Summarize:

- Proposed solution
- Major findings
- Achievement of requirements
- Limitations
- Possible improvements

9. Student Reflection

- What did you learn from this assignment beyond what was directly taught in the classroom?
- If you were given additional time/resources, what would you improve and why?

10. References

Use an appropriate citation format and acknowledge all external sources, datasets, standards, software and AI-assisted tools used.

ANALYTICAL STUDY AND IMPLEMENTATION OF BRUTE FORCE ALGORITHMS FOR SELECTION SORT AND SEQUENTIAL SEARCH

1. ABSTRACT

Sorting and searching are fundamental operations in computer science and are used in applications such as databases, e-commerce, banking, inventory management, data analytics and information retrieval.

This assignment studies two **Brute Force algorithms**:

1. **Selection Sort** – for sorting a collection of numbers.
2. **Sequential Search** – for searching for a particular value.

The given dataset is:

$A=\{42,17,8,93,26,55,11,68,34,5,79,21\}$

Selection Sort repeatedly finds the smallest element from the unsorted portion and places it in the correct position. Sequential Search checks elements one by one from the beginning until the required key is found.

For the given input, Selection Sort produces:

$\{5,8,11,17,21,26,34,42,55,68,79,93\}$

The total number of Selection Sort comparisons is:

66

The number of actual swaps is:

9

For Sequential Search, the key **68** is found at:

- **Index:** 7
- **Position:** 8
- **Comparisons:** 8

The study also examines the scalability limitations of brute-force methods and compares them with efficient approaches such as Merge Sort, Quick Sort, Hybrid Sort and Binary Search.

2. INTRODUCTION

Algorithms provide systematic methods for solving computational problems.

Two of the most common operations performed on data are:

Sorting

Arranging data in a particular order, usually ascending or descending.

Searching

Finding a particular element or key within a collection of data.

In this assignment, brute-force techniques are studied because they provide simple and direct solutions.

The assignment requires:

- Selection Sort
- Sequential Search
- Step-by-step execution
- Pseudocode
- Comparison counting
- Swap counting
- Complexity analysis
- Experimental comparison
- Efficient alternatives
- Graphical analysis
- Industry relevance
- Limitations and improvements
- SDG relevance

These requirements are explicitly specified in the assignment document.

3. PROBLEM STATEMENT

Given:

$A = \{42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21\}$ $A = \{42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21\}$

perform the following operations:

Part A – Selection Sort

Apply Selection Sort using the brute-force approach.

Required:

- Show intermediate steps.
- Find the final sorted sequence.
- Count comparisons.
- Count swaps.
- Develop pseudocode.
- Analyze complexity.

Part B – Sequential Search

Search for:

Key=68Key = 68

using Sequential Search.

Required:

- Show each comparison.
- Identify the position of 68.
- Count comparisons.
- Analyze best, average and worst cases.

The assignment specifically asks for these operations and their comparison as input size increases.

4. OBJECTIVES

The objectives are:

1. Understand the Brute Force technique.
2. Implement Selection Sort.

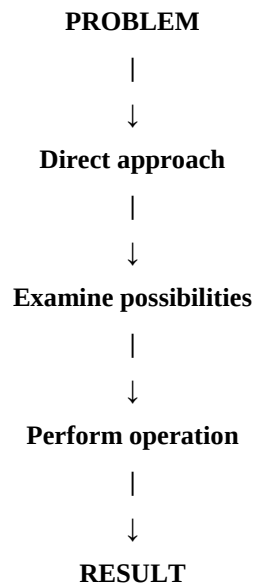
3. Implement Sequential Search.
4. Perform manual step-by-step execution.
5. Count comparisons and swaps.
6. Derive Selection Sort's comparison formula.
7. Analyze Big-O, Big- Ω and Big- Θ .
8. Study scalability.
9. Identify limitations of brute-force algorithms.
10. Compare brute-force algorithms with efficient alternatives.
11. Study practical industry applications.
12. Understand the relationship between algorithm efficiency and real-world systems.
13. Relate the project to SDG 4 and SDG 9.

5. BRUTE FORCE TECHNIQUE

Definition

Brute Force is a straightforward problem-solving technique that solves a problem by systematically examining possible choices or directly following the problem definition without using advanced optimization.

General Brute Force Structure



Advantages

- Simple to understand
- Easy to implement
- Easy to debug
- Useful for small datasets
- Useful as a baseline algorithm

Disadvantages

- Usually inefficient for large datasets
- May perform many unnecessary operations
- Does not exploit advanced data structures or optimization techniques

6. SELECTION SORT

6.1 Definition

Selection Sort is a comparison-based sorting algorithm.

It divides the array into:

SORTED PORTION | UNSORTED PORTION

During every pass:

1. Find the smallest element in the unsorted portion.
2. Swap it with the first element of the unsorted portion.
3. Increase the sorted portion.
4. Repeat until the array is sorted.

7. SELECTION SORT DIAGRAM

Pass	Minimum Selected	Swap	Array After Pass
Initial	—	—	42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21
1	5	42 ↔ 5	5, 17, 8, 93, 26, 55, 11, 68, 34, 42, 79, 21
2	8	17 ↔ 8	5, 8, 17, 93, 26, 55, 11, 68, 34, 42, 79, 21
3	11	17 ↔ 11	5, 8, 11, 93, 26, 55, 17, 68, 34, 42, 79, 21
4	17	93 ↔ 17	5, 8, 11, 17, 26, 55, 93, 68, 34, 42, 79, 21
5	21	26 ↔ 21	5, 8, 11, 17, 21, 55, 93, 68, 34, 42, 79, 26
6	26	55 ↔ 26	5, 8, 11, 17, 21, 26, 93, 68, 34, 42, 79, 55
7	34	93 ↔ 34	5, 8, 11, 17, 21, 26, 34, 68, 93, 42, 79, 55
8	42	68 ↔ 42	5, 8, 11, 17, 21, 26, 34, 42, 93, 68, 79, 55
9	55	93 ↔ 55	5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93
10	68	No swap	5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93

11	79	No swap	5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79 , 93
----	----	---------	--

Final Answer

[5,8,11,17,21,26,34,42,55,68,79,93]

8. SELECTION SORT ALGORITHM

Step-by-step algorithm

Step 1: Start.

Step 2: Set $i = 0$.

Step 3: Assume $A[i]$ is the minimum.

Step 4: Compare it with every remaining element.

Step 5: If a smaller element is found, update min_index .

Step 6: Swap $A[i]$ with $A[\text{min_index}]$.

Step 7: Increment i .

Step 8: Repeat until $i = n-2$.

Step 9: Stop.

9. SELECTION SORT PSEUDOCODE

```

SelectionSort(A, n)

for i ← 0 to n-2
    min_index ← i
    for j ← i+1 to n-1
        if A[j] < A[min_index]
            min_index ← j
    if min_index ≠ i
        swap A[i] and A[min_index]

return A

```

10. MANUAL EXECUTION OF SELECTION SORT

Given:

42 17 8 93 26 55 11 68 34 5 79 21

Pass 1

Find smallest from the complete array.

Smallest = 5

Swap 42 and 5.

5 17 8 93 26 55 11 68 34 42 79 21

Pass 2

Unsorted portion:

17 8 93 26 55 11 68 34 42 79 21

Smallest = 8

5 8 17 93 26 55 11 68 34 42 79 21

Pass 3

Unsorted portion:

17 93 26 55 11 68 34 42 79 21

Smallest = 11

5 8 11 93 26 55 17 68 34 42 79 21

Pass 4

Smallest = 17

5 8 11 17 26 55 93 68 34 42 79 21

Pass 5

Smallest = 21

5 8 11 17 21 55 93 68 34 42 79 26

Pass 6

Smallest = 26

5 8 11 17 21 26 93 68 34 42 79 55

Pass 7

Smallest = 34

5 8 11 17 21 26 34 68 93 42 79 55

Pass 8

Smallest = 42

5 8 11 17 21 26 34 42 93 68 79 55

Pass 9

Smallest = 55

5 8 11 17 21 26 34 42 55 68 79 93

Pass 10

Smallest = 68

Already in correct position.

5 8 11 17 21 26 34 42 55 68 79 93

No swap.

Pass 11

Smallest = 79

Already in correct position.

5 8 11 17 21 26 34 42 55 68 79 93

No swap.

11. SELECTION SORT RESULT

Final sorted array

5,8,11,17,21,26,34,42,55,68,79,93

12. NUMBER OF COMPARISONS

For Selection Sort:

Pass	Comparisons
1	11
2	10
3	9
4	8
5	7
6	6
7	5
8	4
9	3
10	2
11	1
Total	66

Therefore:

$11+10+9+8+7+6+5+4+3+2+1+11+10+9+8+7+6+5+4+3+2+1 = 66=66$

General formula:

$$(n-1)+(n-2)+\dots+1 = 2n(n-1)$$

For $n=12$:

$$(12*11)/2 = 66$$

Therefore:

66 comparisons

13. SWAP COUNT

Selection Sort performs an actual swap only when the minimum element is not already at the current position.

For the given input:

Pass	Swap
1	Yes
2	Yes
3	Yes
4	Yes
5	Yes
6	Yes
7	Yes
8	Yes
9	Yes
10	No
11	No

Therefore:

9 actual swaps

14. SELECTION SORT COMPLEXITY

Time Complexity

Case	Complexity
Best Case	$\Theta(n^2)$
Average Case	$\Theta(n^2)$
Worst Case	$\Theta(n^2)$

Space Complexity

$O(1)$

15. WHY SELECTION SORT BECOMES SLOW

The number of comparisons is:

$$C(n) = n(n-1)/2$$

n	Comparisons
10	45
100	4,950
1,000	499,500
10,000	49,995,000
100,000	4,999,950,000
1,000,000	499,999,500,000

The curve rises rapidly because:

$$T(n) \propto n^2$$

16. SEQUENTIAL SEARCH

Definition

Sequential Search, also called Linear Search, examines each element one by one until:

- the required element is found, or
- the entire array is checked.

It is a direct brute-force searching technique.

17. SEQUENTIAL SEARCH DIAGRAM

Searching for:

Key=68

```

42 → 17 → 8 → 93 → 26 → 55 → 11 → 68
↑   ↑   ↑   ↑   ↑   ↑   ↑   ↑
No  No  No  No  No  No  No  FOUND

```

Therefore:

```

68 is found at index 7
68 is at position 8

```

18. SEQUENTIAL SEARCH ALGORITHM

Steps

1. Start from the first element.
2. Compare the current element with the key.
3. If equal, return the position.
4. Otherwise move to the next element.
5. Continue until the key is found.
6. If the end is reached, return "Not Found".

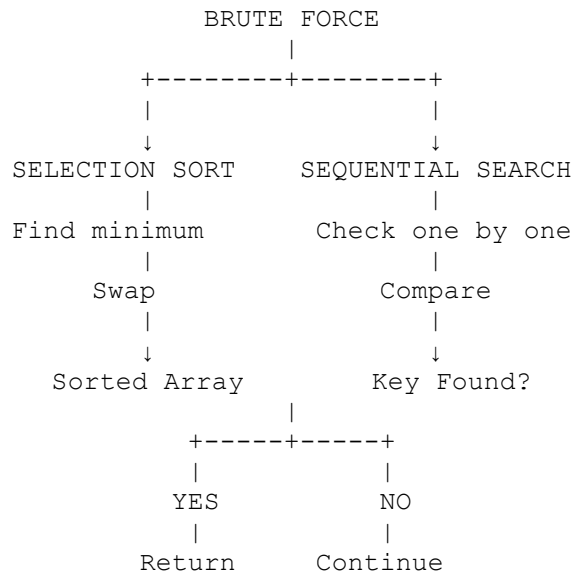
19. SEQUENTIAL SEARCH PSEUDOCODE

SequentialSearch(A, n, key)

```
for i ← 0 to n-1
    if A[i] = key
        return i
```

```
return -1
```

20. COMPLETE ALGORITHM COMPARISON



21. PYTHON IMPLEMENTATION

A = [42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21]

```
def selection_sort(arr):
```

```
    a = arr.copy()
```

```
    comparisons = 0
```

```
    swaps = 0
```

```
    print("\n--- SELECTION SORT ---")
```

```
    print("Initial:", a)
```

```
    for i in range(len(a) - 1):
```

```
        min_index = i
```

```
        for j in range(i + 1, len(a)):
```

```
            comparisons += 1
```

```
            if a[j] < a[min_index]:
```

```
                min_index = j
```

```
    if min_index != i:
```



```

        a[i], a[min_index] = a[min_index], a[i]

        swaps += 1

    print(f"Pass {i + 1}: {a}")

print("Sorted array:", a)

print("Comparisons:", comparisons)

print("Swaps:", swaps)

return a

def sequential_search(arr, key):
    comparisons = 0

    print("\n--- SEQUENTIAL SEARCH ---")

    for i in range(len(arr)):
        comparisons += 1

        print(f"Comparison {comparisons}: {arr[i]}")

        if arr[i] == key:
            print("Key found!")

            print("Position:", i + 1)

            print("Index:", i)

            print("Comparisons:", comparisons)

            return i

    print("Key not found.")

    print("Comparisons:", comparisons)

    return -1

print("=" * 50)

print("BRUTE FORCE SORTING AND SEARCHING")

print("=" * 50)

print("Given array:")

print(A)

sorted_array = selection_sort(A)

sequential_search(A, 68)

```

22. INDUSTRY RELEVANCE

Sorting and searching are not merely academic operations. They form the foundation of many information-processing systems.

Examples include:

Industry	Sorting Application	Searching Application
E-commerce	Product price/rating sorting	Product ID search
Banking	Transaction organization	Account/transaction lookup
Healthcare	Patient record organization	Patient ID lookup
Logistics	Shipment prioritization	Tracking ID search
Education	Student ranking	Student ID search
Inventory	Product ordering	Stock/item lookup
Data Analytics	Dataset ordering	Record lookup
Social Media	Feed/ranking operations	User/content lookup

23. COMPARISON OF ALGORITHMS

Algorithm	Purpose	Best	Average	Worst
Selection Sort	Sorting	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$
Merge Sort	Sorting	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$
Quick Sort	Sorting	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n^2)$
Sequential Search	Searching	$\Omega(1)$	$\Theta(n)$	$O(n)$
Binary Search	Searching	$\Omega(1)$	$\Theta(\log n)$	$O(\log n)$

24. PROJECT IMPROVEMENT STRATEGY

BRUTE FORCE



Implement Algorithm



Measure Operations



Analyze Complexity



Identify Bottleneck



Compare Alternatives



Select Efficient Method

25. REAL-WORLD SYSTEM EXAMPLE – E-COMMERCE

Consider an e-commerce website containing millions of products.

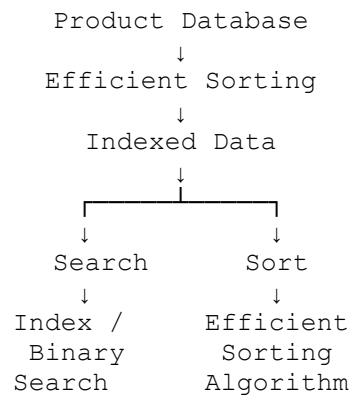
Problem

Products need to be:

- Sorted by price
- Sorted by rating
- Searched by product ID
- Filtered by category

Using brute-force techniques repeatedly would become inefficient.

Improved architecture



This illustrates why algorithm selection matters in industry.

26. SDG MAPPING

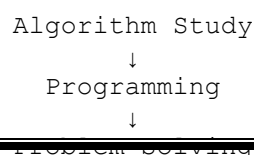
SDG 4 – QUALITY EDUCATION

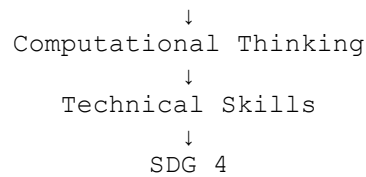
The project develops:

- Algorithmic thinking
- Programming skills
- Mathematical reasoning
- Computational problem-solving
- Technical knowledge

UN SDG Target 4.4 specifically focuses on increasing relevant technical and vocational skills, including ICT skills.

Project connection

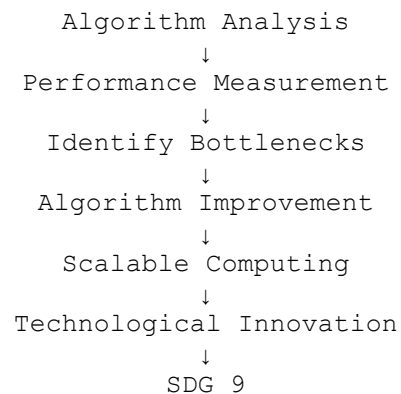




SDG 9 – INDUSTRY, INNOVATION AND INFRASTRUCTURE

SDG 9 focuses on resilient infrastructure, sustainable industrialization and innovation. It also includes strengthening technological capabilities and innovation.

Project connection



The project therefore has a **stronger direct connection to SDG 4 and SDG 9**, which are also the SDGs explicitly mapped in the assignment.

27. CONCLUSION

This assignment successfully demonstrates two fundamental brute-force algorithms: **Selection Sort** and **Sequential Search**.

Selection Sort was applied to the given 12-element dataset and produced the sorted sequence:

5,8,11,17,21,26,34,42,55,68,79,93

A total of:

66

comparisons and:

9

actual swaps were performed.

Sequential Search was then used to find key **68**. The algorithm found the key at the **8th position**, corresponding to **index 7**, after:

8 comparisons

The analysis shows that Selection Sort has:

$\Theta(n^2)$

time complexity, while Sequential Search has:

$$O(n)$$

worst-case complexity.

Although these algorithms are simple and useful for understanding fundamental algorithmic concepts, their brute-force nature makes them unsuitable for many large-scale applications. More efficient methods such as Merge Sort, Quick Sort, Hybrid Sort and Binary Search provide better scalability.

G. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	10	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	15	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	20	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates a limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.

		identified problem.			
Implementation & Modern Tool Usage	15	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable, or non-functional.
Efficiency, Testing & Validation	15	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement, Comparison & Final Decision	15	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	Good performance comparison and justification with minor limitations.	Basic comparison is provided, but improvement or final justification lacks depth.	Improvement is not demonstrated or the final algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements,	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithmic development, SDG relevance, challenges,

		trade-offs, and learning gained.			or learning outcomes.
--	--	----------------------------------	--	--	-----------------------

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results
3. Github Upload